Inherited foundations for concise iPRA

Attributed companion source compilation

ProveIt research sources; compilation prepared for Vladimir Reshetnikov

November 11, 2026

This companion supplies the existing mathematical results used by the main report. The new target-directed rule and lexical validation theorem are in *Target-directed closure and smaller typed checking for iPRA*; they are not claimed as results of the historical sections reproduced here.

The source snapshot is ProveIt commit `76697262071ada91f29c14d9cf3801a64dacb994`, directory `Logic/PrimitiveRecursiveArithmetic/ConciseIPRA`. The five scalar foundation modules have been checked byte-for-byte against their Git blob identities in `Research/certified_zero/reference/`; the receipt is `evidence/foundation_identity.json`. The zero-source theorem was retrieved directly from the pinned `source_dispatch` package. The progressive closure sections are reproduced from the incoming `ipra_uniform_progress_2026-11-11.zip`, whose Git blob is `2b8392d6036daf0272b8697b549557aefc9e1cad`. The upstream MIT No Attribution license is retained in `reference/LICENSE-ProveIt.txt`.

Historical phrases such as “new”, “current”, and “the new calculus” in reproduced sections refer to their original reports. In particular, $\mathcal{R}$ in the progressive part is the inherited orbit calculus, not the target-directed $\mathcal{T}$ of the main report. Its forward translation is called T in the historical text and F in the main report. The native conditional theory is a different theory with the uniform reading $QB = B(0) \to \forall y B(y)$.

This is a compilation of written proofs, not a fresh verification of the upstream Lean project. The full-language progressive roundtrip theorem is an inherited written theorem; the native/scalar/iPRA bridge also has upstream Lean endpoint files, which were inspected but not rebuilt here.

Contents

I Conditional native logic and scalar arithmetic 3

1 Complementary introductions with one local assumption 3

2 Internal scalar predecessor bootstrap 8

3 From literal-equation induction to full intuitionistic PRA 12

4 Pairing, primitive recursion, and internal numerical composites 17

5 Arithmetic strength and both ordinary interpretation composites 19

II The source restriction used by the compact checker 21

6 Removing literal zero from the source of transport 21

III The inherited progressive orbit calculus and its full round trips 27

7 A seven-rule calculus with complete progressive elimination 27

8 Explicit logical macros in the reference language 34

9 The arithmetic interface and the inherited bridge 35

Part I

Conditional native logic and scalar arithmetic

1 Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

1.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}\ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F}\ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}\ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x, y)}\ \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In

transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t,Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t,z)$ exactly at a literal equation.

1.2 Exact equivalence and the unused-witness construction

Lemma 1.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem 1.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x,y)$ and its transformed premise is

$$\pi: \quad \Gamma, E(t,0), E(t,z) \vdash E(t,Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t,Sz)$ under $\Gamma, E(t,0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition 1.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

1.3 Induction and numerical cases separate cleanly

Lemma 1.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma 1.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0,t;u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

1.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v,a,b) := I(v,a;u.b)$ with u unused. Computation gives $C(0,a,b) = a$ and $C(Sv,a,b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s=t) := (C(v,0,s) = C(v,0,t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u,y \notin \mathrm{FV}(A)). \qquad \text{(Gate and padding)}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s=t, u=w) := (C(v,s,u) = C(v,t,w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A,B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE,B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A,G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A,B) \dashv\vdash A$ and $M_{Sv}(A,B) \dashv\vdash B$.

Put $\delta(v) = C(v,1,0)$ and $p(z) = I(z,0;u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A,B),\\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A,B),\\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x),\\ A \vee_* B &:= \exists_* x\, M_x(A,B),\\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z),\\ \bot_* &:= (0=1). \end{aligned} \qquad \text{(Logical macros)}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A,B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A,B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\,(x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

1.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \tag{Paired composite}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both

formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem 1.2 preserves every native sequent.

1.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

2 Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [4] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

2.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved

by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

2.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n) + 1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

2.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

2.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \qquad (12)$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \qquad (13)$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \qquad (14)$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \qquad (15)$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

2.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

3 From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [5]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix 2, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

3.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x\,(x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p}\,\left[(E(0, \vec{p}) \wedge \forall n(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n\, E(n, \vec{p})\right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

3.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$

$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$

$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

3.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S((Sx) + y) = S(S(x + y)) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

3.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}(K(d(Sa, b))). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, (a = b \vee (a = b \to \bot)) \tag{41}$$

is a theorem, not an extra hypothesis.

3.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{42}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{43}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{44}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

3.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

4 Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [6]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

4.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \leq b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a+b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a+b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

4.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

4.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it

with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section 1 give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

5 Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section 1 recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

5.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.
4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n, \vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

Equation induction proves $L(V(n, \vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem 5.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

Part II
The source restriction used by the compact checker

6 Removing literal zero from the source of transport

The following is a restriction of a mathematical inference rule, rather than a change in the spelling of zero. Throughout this section, "nonzero source" means *not syntactically the single term* 0. Every variable and every term headed by S or I is a permitted source, including terms whose numerical value is zero.

Let $\mathcal{C}$ be the native scalar calculus with terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h),$$

the two computation schemas

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u],$$

formulas generated by equality and the Q, D binders, and the Hyp, QE, DE, case-DI and equation-progress-DI rules stated in the main inventory. The same argument applies to the original presentation with D object formulas and Q resources in contexts. Let $\mathcal{C}^-$ replace unrestricted transport by

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \;\mathsf{Tr}^-, \qquad s \not\equiv 0. \tag{Tr$^-$}$$

Here F is an arbitrary native numerical template, and $s \not\equiv 0$ is a condition on finite term syntax. All other rules are unchanged.

Theorem 6.1 (Literal-zero source elimination)**.** *For every native context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{C}^-} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}} A.$$

In particular, the ordinary whole-domain realization of $\mathcal{C}^-$ *is bi-interpretable and equiconsistent with full first-order iPRA through the same interpretations as the native scalar core. No change of numerical domain or interpreted equality is needed.*

We prove the theorem without assuming arbitrary numerical substitution admissibility for $\mathcal{C}^-$: such substitution can turn a variable source into the forbidden literal zero. Every occurrence of a helper equation at an arbitrary term is either obtained directly from its displayed derived rule at that term, or from a derivation whose transport sources retain an S or I head under the indicated substitution.

6.1 Structural operations, cases, and equation induction

Weakening, ordinary-assumption cut, and capture-free renaming are admissible by induction on derivations. These operations do not turn a nonzero source into literal zero. As usual, fresh parameters are renamed before a cut. We also use the following restricted substitution fact: a capture-free substitution is admissible for a particular finite derivation if it leaves every primitive transport source in that derivation syntactically different from zero. This fact follows by the same induction; it is not unrestricted substitution admissibility.

Lemma 6.2 (Cases and equation induction need no transport)**.** *In $\mathcal{C}^-$ the following rules are derivable:*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma \vdash A(Sz)}{\Gamma \vdash A(t)}\ \mathsf{NC}, \qquad \frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}\ \mathsf{EI}.$$

In NC, A is any native formula; in EI, E is one literal equation. The parameter z is fresh for Γ, t, and the fixed parameters of the matrix. Other parameters stay fixed throughout EI.

Proof. Choose an unused witness coordinate x. In NC, weaken the successor proof by $A(0)$ and apply case-DI, with witness zero, to obtain $D_{x,y}A(y)$. DE opens $Q_yA(y)$, because x is unused. QE with the given base proof and target t yields $A(t)$, and DE closes. For EI use progress-DI instead, with the given equation step, and use the same DE/QE construction. No transport rule occurs. □

These rules will be applied to unconditional helper equations before they are weakened into the context containing $0 = t$. We never use NC with a case parameter occurring in its retained context.

6.2 The equality operations available immediately

Lemma 6.3 (Reflexivity and safe equality operations)**.** *Reflexivity $r = r$ is derivable for every term r. From $p = q$ with $p \not\equiv 0$, one can derive $q = p$ and $H(p) = H(q)$ for any numerical template H. Every direct computation can be used in either direction as an equation; using the reversed equation as a transport major is permitted only when its new source is syntactically nonzero.*

Proof. Put $b = I(0, r; u.0)$. The computation $b = r$, used as both major and minor of transport with template $v = r$, gives $r = r$. Its source is the nonempty iterator term b. For symmetry of $p = q$, transport the reflexivity $p = p$ through the template $v = p$. For numerical congruence, transport the reflexivity $H(p) = H(p)$ through $H(p) = H(v)$. Both use the permitted source p. Every computation equation has an I-headed left side, so its symmetry follows by the same construction. □

An occurrence-selective template may keep some occurrences of a term fixed while abstracting other occurrences. This is ordinary capture-free template substitution, not an extra inference rule.

Lemma 6.4 (Joining two nonzero terms known to be zero)**.** *Suppose $p, q \not\equiv 0$ and Γ proves $p = 0$ and $q = 0$. Then $\Gamma \vdash p = q$ using only Tr^- and computation.*

Proof. Let

$$z_0 = I(0, 0; u.0), \qquad b_p = I(0, p; u.0).$$

Computation gives $b_p = p$. Start with $b_p = b_p$ and use the major $p = 0$ in the template $I(0, v; u.0) = b_p$. This gives $z_0 = b_p$. Symmetry at the nonzero source z_0 gives $b_p = z_0$. Now transport through $v = z_0$ using the computation $b_p = p$, whose source is b_p, to obtain $p = z_0$. The identical construction gives $q = z_0$. Symmetry at source q gives $z_0 = q$, and transport at source z_0 in $p = z_0$ gives $p = q$. The sources used are p, q, z_0, b_p, b_q; all are syntactically nonzero. No reverse transport from zero has been used. □

The zero-join lemma is deliberately placed before the next construction. The positive case of its singleton test needs precisely this lemma; ordinary transitivity through zero is not yet available.

6.3 A singleton-one test without predecessor

Define the finite scalar terms

$$\begin{aligned} C(v,a,b) &:= I(v,a;u.b), & Z(v) &:= C(v,1,0),\\ K(v) &:= C(v,0,1), & H(v) &:= C(v,1,2),\\ N(n) &:= I(n,0;u.Z(u)), & R(n) &:= I(n,0;u.H(u)),\\ L(n) &:= N(R(n)). \end{aligned} \tag{Tests}$$

The update variable in C is unused. Here $1 = S0$ and $2 = S1$. The standard-model description of R is $0, 1, 2, 2, \ldots$; the proof below establishes the needed equations internally and does not assume this description.

Lemma 6.5 (Uniform test equations)**.** *The following equations have derivations in $\mathcal{C}^-$, uniformly in the displayed argument:*

$$H(v) = SK(v), \qquad R(1) = 1, \qquad R(SSv) = 2,$$

$$N(1) = 1, \quad N(2) = 0, \quad L(1) = 1, \quad L(SSv) = 0, \quad \boxed{L(Sv) = Z(v)}. \tag{Singleton}$$

All transport sources in these helper derivations have an I or S head, except for the explicit nonzero endpoints in the zero-join lemma; those endpoints also have an I head in this application.

Proof. For $H(v) = SK(v)$ use NC. At zero both sides reduce to 1; at a successor both reduce to 2. To join two equations $p = j$, $q = j$ with $j = 1$ or $j = 2$, reverse $q = j$ at its nonzero source q, then transport at the nonzero numeral j. Each reduction of these particular terms is a forward replacement of an iterator computation. This proves the helper at an arbitrary target v by NC, without substituting zero into a variable-source transport.

The equations $N(1) = 1$ and $N(2) = 0$ are finite computations: $N(0) = 0$, $N(1) = Z(N(0)) = 1$, and $N(2) = Z(N(1)) = 0$. Every replacement source in these calculations is an iterator term. Likewise $R(1) = H(R(0)) = H(0) = 1$.

Computation and the first helper give, for any v,

$$R(Sv) = H(R(v)) = SK(R(v)).$$

Use this equation at the nonzero source $R(Sv)$ in $R(SSv) = H(R(Sv))$. The resulting right side is $H(SK(R(v))) = 2$ by the successor computation of C. Thus $R(SSv) = 2$. These substitutions are all at I-headed sources. Substitute $R(1) = 1$ and $R(SSv) = 2$ inside N and use the two computed values of N to obtain $L(1) = 1$ and $L(SSv) = 0$.

Finally apply NC to the matrix $L(Sv) = Z(v)$. At zero, $L(1) = 1$ and $Z(0) = 1$ are joined through the nonzero numeral 1. At a successor, $L(SSv) = 0$ and $Z(Sv) = 0$ are joined by Lemma 6.4; both left sides are nonempty iterator terms. This proves the boxed equation at every target. Its use is therefore an instance of a displayed derived rule, not an appeal to unrestricted substitution admissibility. □

Lemma 6.6 (An unconditional zero-filter identity)**.** *For every term t,*

$$t = C(Z(t), t, 0) \tag{Filter}$$

is derivable in the empty context of $\mathcal{C}^-$.

Proof. Apply NC to $v = C(Z(v), v, 0)$, with the target t. At zero, forward computations give $C(Z(0), 0, 0) = C(1, 0, 0) = 0$; reverse this equation at its nonzero left side. At Sv, forward computations give $C(Z(Sv), Sv, 0) = C(0, Sv, 0) = Sv$; again reverse at the nonzero iterator left side. Thus the base and successor proofs have no retained assumption depending on the case variable. All numerical arguments of the final NC target are permitted. In particular, this identity is proved before any assumption $0 = t$ is used. □

6.4 Turning a zero-source equality around

Lemma 6.7 (Zero-source symmetry)**.** *For every term t,*

$$0 = t \ \vdash_{\mathcal{C}^-} \ t = 0. \tag{ZS}$$

Proof. If $t \equiv 0$, use reflexivity. Otherwise t is a permitted source. Work in the fixed context $\Gamma = \{0 = t\}$ and put

$$A(n) := I(n, t; u.Su).$$

We first prove $n = A(n)$ by EI, keeping t fixed.

For the base, computation gives $A(0) = t$. Its symmetry gives $t = A(0)$. Since $t \not\equiv 0$, transport at source t in the retained assumption $0 = t$ gives $0 = A(0)$.

For the step, retain $z = A(z)$ with z fresh. This source is a variable, hence is permitted. Numerical successor congruence from Lemma 6.3 gives $Sz = SA(z)$. Computation gives $A(Sz) = SA(z)$, and its symmetry gives $SA(z) = A(Sz)$. Its source $SA(z)$ is also permitted, so transport in $Sz = SA(z)$ gives $Sz = A(Sz)$. This is exactly the EI step. The induction hypothesis is used at the same fixed parameter t.

At target 1, EI gives $1 = A(1)$. Forward computation gives $A(1) = St$: the successor equation produces $SA(0)$, and the nonzero-source computation $A(0) = t$ is used inside S. Transport at source $A(1)$ consequently gives

$$1 = St.$$

Its source 1 is permitted. Numerical congruence for L gives $L(1) = L(St)$. Use $L(1) = 1$ at source $L(1)$, and $L(St) = Z(t)$ at source $L(St)$, obtaining $1 = Z(t)$. Symmetry at source 1 gives $Z(t) = 1$.

Weaken the unconditional Filter identity into Γ. Transport at the iterator source $Z(t)$ changes its right side to $C(1, t, 0)$. The latter computes to zero, and its source is again an iterator term. The resulting equation is $t = 0$, as required. □

This proof uses cases only for unconditional helper equations. The sole induction with the retained assumption $0 = t$ has the literal equation $n = A(n)$ as invariant. It does not perform a forbidden case distinction while silently changing the retained context.

6.5 Recovering equality algebra and formula transport

Lemma 6.8 (Full numerical equality algebra)**.** *The calculus $\mathcal{C}^-$ admits unrestricted symmetry, numerical congruence, and transitivity. In particular it admits root replacement*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash s = r}{\Gamma \vdash t = r}.$$

Proof. Symmetry uses Lemma 6.3 when the source is not zero and Lemma 6.7 otherwise.

For numerical congruence from $s = t$, a nonzero s is handled by Lemma 6.3. If $s \equiv t \equiv 0$, use reflexivity. If $s \equiv 0$ and $t \not\equiv 0$, first derive $t = 0$, apply permitted numerical congruence at source t to obtain $H(t) = H(0)$, and use the now established full symmetry.

For transitivity from $r = s$ and $s = t$, a nonzero middle term s is handled by one permitted transport. If $s \equiv 0$ and either endpoint is literally zero, a given premise is already the conclusion. Otherwise $r, t \not\equiv 0$; reverse $0 = t$ to get $t = 0$ and apply the zero-join lemma to $r = 0$ and $t = 0$. Root replacement follows by symmetry and transitivity. These constructions are acyclic: zero-source symmetry was proved before any unrestricted numerical congruence or transitivity. □

Lemma 6.9 (Full transport)**.** *For every native numerical template $F(v)$, the rule*

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}$$

is admissible in $\mathcal{C}^-$, without a source restriction.

Proof. Use external induction on the finite number of formula binders in F, uniformly in retained contexts and fixed numerical parameters. For an equation $F(v) \equiv A(v) = B(v)$, use unrestricted numerical congruence to obtain $A(s) = A(t)$ and $B(s) = B(t)$. Join these with $A(s) = B(s)$ using the established equality algebra.

For $F(v) = D_{x,y}B(v, x, y)$, eliminate $F(s)$ at a fresh a, retaining $Q_yB(s, a, y)$. Introduce the desired $F(t)$ by case-DI with witness a. Its local assumption is $B(t, a, 0)$. Full symmetry gives $t = s$, and the external induction hypothesis for the smaller matrix transports the assumption to $B(s, a, 0)$. QE gives $B(s, a, Sz)$, and the induction hypothesis in the forward direction gives $B(t, a, Sz)$. Close case-DI and DE. All new parameters can be chosen fresh for the source equality and the retained context.

If Q is first-class formula syntax, first observe that its case introduction is derived without transport: introduce $D_{u,y}B(y)$ with unused u by case-DI and eliminate it with the one-node continuation $Q_yB(y)$. For $F(v) = Q_yB(v, y)$, use this derived introduction for the target. Convert its base assumption backwards by the induction hypothesis, apply the old Q resource, and convert the successor conclusion forwards. This is the same smaller-matrix argument and uses no new numerical induction. □

Proof of Theorem 6.1. Every restricted derivation is an unrestricted one. Conversely, replace each unrestricted transport node by the derivation supplied by Lemma 6.9, recursively translating its premises first. Other inference nodes are unchanged. The result has the same context and conclusion. Thus the full native consequence relation, not just closed theorems, is preserved. The existing native/ordinary and ordinary/iPRA interpretations and both identity composites therefore transfer unchanged. □

6.6 Compatibility with restricted computation certificates

Some compact checkers require the initializer of a computation packet to have nonempty syntax, and encode a successor counter by a nonempty predecessor. These are certificate restrictions, not restrictions on the two mathematical computation schemas. They do not introduce a circular dependence on zero-source transport in the preceding proof.

Use the inherited certified-zero term

$$c_0 = I(2, 1; u.0).$$

Its equation $c_0 = 0$ is a directly admitted successor computation: the predecessor and initializer are both 1. For an arbitrary computation instance with literal-zero initializer or literal-zero successor predecessor, form its whole equation as a template $J(v)$, replacing just these offending occurrences by v. The instance $J(c_0)$ is a directly admitted computation. Transport through this equation template using $c_0 = 0$ yields $J(0)$, the desired original instance. Both computation premises are admitted directly, and the only transport source is the nonempty term c_0.

This is exactly the inherited three-node normalization in `Research/certified_zero/tests/producer.py`, function `expand_computation`: one transport and two computation premises. Its source is c_0, even when both defects are corrected simultaneously. Passive zero occurrences in an update are left alone. Thus all computation instances used in the mathematical proof can be expanded before compiling to the source-restricted checker.

6.7 Scope of the result

The new statement is the admissibility of all omitted literal-zero source transports. The native term/formula language, arithmetic schemas, and target theory remain the inherited ones. This theorem justifies replacing a two-way equality/Q major-pattern test by a source-discriminated test in a suitable certificate protocol. It does not assert a minimum proof-checker size, a shortest proof expansion, or a new characterization of the primitive recursive functions. The displayed proof is a written universal derivation; executable certificates, when supplied, validate particular instances rather than substitute for this argument.

Part III

The inherited progressive orbit calculus and its full round trips

7 A seven-rule calculus with complete progressive elimination

The new presentation keeps the scalar iterator and the three native formula constructors. It changes the meaning of an atomic closure: an atomic closure records a progressive equation, while a compound closure retains the old conditional-universal meaning. A single elimination rule recovers both one-step progress and induction from an atomic closure. Consequently every formula of the new language has a faithful ordinary reading; no restricted public grammar or auxiliary-only formula class is needed.

This is a structural simplification, not a claim of an absolute minimum. The current Lean kernel has eight rule constructors. The proposed kernel has seven, and its introduction rule commutes directly with numerical substitution. The earlier seven-rule `WPrf` presentation also has seven constructors, but its introduction distinguishes the literal witness 0 from other witnesses. Here the distinction depends only on whether a matrix is a literal equation, which numerical substitution cannot change. The concrete checker measurements, including comparisons in which this calculus is larger, are reported separately.[1, 8]

7.1 Complete primitive specification

There is one numerical sort. Terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u is bound in h; other occurrences are free parameters. Substitution is capture-free. Put

$$1 := S0, \qquad \mathsf{A}(a, n) := I(a, n; u.Su). \tag{47}$$

Thus the first argument of A is the iteration counter and the second is its initializer. This argument order is essential below.

Formulas are

$$F ::= (s = t) \mid Q_y F \mid D_{x,y} F. \tag{48}$$

The binder Q binds y, and D binds the distinct variables x, y. A *literal equation* has outermost constructor $=$. A *compound* formula has outermost constructor Q or D. Define the syntactic selector

$$\eta_B(a) := \begin{cases} a, & B \text{ is a literal equation,} \\ 0, & B \text{ is compound.} \end{cases} \tag{49}$$

Write $\Gamma \vdash_{\mathcal{R}} F$ for derivability. Contexts are finite reusable lists of formulas. The complete primitive inventory is the following seven rule schemas:

$$\frac{F \in \Gamma}{\Gamma \vdash_{\mathcal{R}} F}\ \text{Hyp},$$

$$\frac{}{\Gamma \vdash_{\mathcal{R}} I(0, a; u.h) = a}\ \text{I0}, \qquad \frac{}{\Gamma \vdash_{\mathcal{R}} I(Sn, a; u.h) = h[I(n, a; u.h)/u]}\ \text{IS},$$

$$\frac{\Gamma \vdash_{\mathcal{R}} s = t \qquad \Gamma \vdash_{\mathcal{R}} p(s) = q(s)}{\Gamma \vdash_{\mathcal{R}} p(t) = q(t)}\ \text{Tr}_{=}, \tag{50}$$

$$\frac{\Gamma \vdash_{\mathcal{R}} Q_y B(y) \qquad \Gamma \vdash_{\mathcal{R}} B(\eta_B(a))}{\Gamma \vdash_{\mathcal{R}} B(\mathsf{A}(\eta_B(a), n))} \quad \mathrm{QE}_{\mathsf{A}}, \tag{51}$$

$$\frac{\Gamma \vdash_{\mathcal{R}} D_{x,y} B(x,y) \qquad \Gamma, Q_y B(b,y) \vdash_{\mathcal{R}} F}{\Gamma \vdash_{\mathcal{R}} F} \quad \mathrm{DE}, \tag{52}$$

$$\frac{\Gamma, B(t, \eta_B(z)) \vdash_{\mathcal{R}} B(t, Sz)}{\Gamma \vdash_{\mathcal{R}} D_{x,y} B(x,y)} \quad \mathrm{DI}_{\eta}. \tag{53}$$

In (50), $p(v), q(v)$ are numerical term templates. In (51), a, n are arbitrary terms in the conclusion's context. In DE, b is fresh for Γ, F and all fixed parameters of B. In DI, z is fresh for Γ, t and all fixed parameters of B. The displayed local assumptions are discharged. The designated contradiction is $0 = 1$.

There are no other primitive axioms, inference rules, induction schemas, logical connectives, successor axioms, or numerical case principles. In particular, equality transport in a compound formula is not primitive.

Lemma 7.1 (Structural rules and substitution)**.** *Weakening, exchange, contraction, renaming, assumption cut, and capture-free numerical substitution are admissible in $\mathcal{R}$.*

Proof. Use induction on a finite derivation, renaming eigenparameters before a substitution or a cut. A cut replaces each relevant hypothesis leaf by the supplied derivation, weakened into its local context. The only new substitution check is

$$(\eta_B(a))\sigma = \eta_{B\sigma}(a\sigma).$$

It holds because numerical substitution preserves the outermost formula constructor. The iterator abbreviation (47) also commutes with substitution. Hence QE and DI translate to instances of the same rules; there is no witness-literal exception to repair. □

7.2 Equality and the induction bootstrap

A useful derived rule is

$$\frac{\Gamma, B(\eta_B(z)) \vdash_{\mathcal{R}} B(Sz)}{\Gamma \vdash_{\mathcal{R}} Q_y B(y)} \quad \mathrm{QI}_{\eta}. \tag{54}$$

To obtain it, apply DI to $D_{x,y}B(y)$ with x unused and witness 0. Apply DE to this result; its local hypothesis is precisely $Q_y B(y)$, which can be exported by Hyp because it is independent of the fresh witness.

Lemma 7.2 (Numerical equality and contextual equation induction)**.** *The usual reflexivity, symmetry, transitivity, and numerical congruence rules are derivable. For every literal equation $E(y)$,*

$$\frac{\Gamma \vdash_{\mathcal{R}} E(0) \qquad \Gamma, E(z) \vdash_{\mathcal{R}} E(Sz)}{\Gamma \vdash_{\mathcal{R}} E(t)} \quad (z \textit{ fresh}) \tag{55}$$

is admissible with arbitrary retained side assumptions Γ.

Proof. Let $J(a) = I(0, a; u.0)$. I0 gives $J(a) = a$; transport this equation along itself with template $v = a$ to obtain $a = a$. The usual equational templates then give symmetry, transitivity, and numerical congruence.

For induction, the step premise and (54) give $Q_y E(y)$. Use (51) with source $a = 0$ and initializer $n = t$. Its conclusion is $E(I(0, t; u.Su))$. I0 and purely equational transport give $E(t)$. No identity $I(t, 0; u.Su) = t$ has been assumed. Using the counter t and initializer 0 in the elimination rule instead would not justify this bootstrap. □

Lemma 7.3 (Elementary iterator identities)**.** *The new calculus proves, with arbitrary parameters,*

$$\begin{aligned} \mathsf{A}(0,n) &= n, & \mathsf{A}(Sa,n) &= S\mathsf{A}(a,n), \\ \mathsf{A}(a,0) &= a, & \mathsf{A}(a,Sn) &= S\mathsf{A}(a,n), \\ \mathsf{A}(a,n) &= \mathsf{A}(n,a), & \mathsf{A}(a,1) &= Sa. \end{aligned} \tag{56}$$

Proof. The first row consists of computation instances. Induct on a for $\mathsf{A}(a,0) = a$. Induct on a again for the initializer-successor identity: its base is $Sn = Sn$, and its step follows by applying S to the induction hypothesis. For commutativity, induct on a. The base is $n = \mathsf{A}(n,0)$; the step is

$$\mathsf{A}(Sa,n) = S\mathsf{A}(a,n) = S\mathsf{A}(n,a) = \mathsf{A}(n,Sa).$$

Finally $\mathsf{A}(a,1) = S\mathsf{A}(a,0) = Sa$. All inductions are instances of (55) on literal equations. □

An atomic closure therefore has the derived one-step elimination

$$Q_y E(y),\ E(a)\ \vdash_{\mathcal{R}}\ E(Sa). \tag{57}$$

Take initializer 1 in QE and use (56). For a compound matrix, QE at source 0 and initializer t has the ordinary conditional-closure effect $Q_y B, B(0) \vdash B(t)$, once transport through B has been derived.

Lemma 7.4 (Full transport and homogeneous binder congruence)**.** *For every formula template $F(v)$,*

$$s = t,\ F(s)\ \vdash_{\mathcal{R}}\ F(t). \tag{58}$$

Moreover, if B and C are uniformly interderivable and have the same root class (both equations or both compound), their Q closures are interderivable, and their D closures are interderivable.

Proof. For transport, induct on F. Equations use the primitive rule. For $Q_y E(v,y)$ with atomic E, introduce the target closure by QI. Reverse-transport its local $E(t,z)$ to $E(s,z)$, use (57) for the source closure, and transport the resulting $E(s,Sz)$ to $E(t,Sz)$.

For $Q_y B(v,y)$ with compound B, introduce the target closure from $B(t,0)$. The induction hypothesis converts this to $B(s,0)$. QE for the source closure at initializer Sz gives $B(s, I(0,Sz; u.Su))$. The induction hypothesis applied inside the strictly smaller matrix B removes the I0 redex and changes the parameter s to t. This proves the QI target $B(t,Sz)$.

For $D_{x,y}B(v,x,y)$, eliminate the source binder at a fresh witness b. Introduce the target at that witness. Convert its local assumption $B(t,b,\eta_B(z))$ to $B(s,b,\eta_B(z))$ by the induction hypothesis. At an atomic matrix use (57); at a compound matrix use QE with source 0, initializer Sz, and I0. Transport the successor result back to the parameter t. Close DI and DE. Every invoked formula transport concerns the strictly smaller matrix, so this is not circular.

For homogeneous congruence, use the same constructions, replacing numerical transport by the two given conversions between B and C. Their root classes agree, so their local DI assumptions use the same value of η. For D, eliminate the source at a fresh witness and introduce the target at that witness. The argument works uniformly under arbitrary additional retained assumptions. □

The root-class qualification is substantive. The new Q is not claimed to preserve arbitrary equivalence when one matrix is an equation and the other is compound. A progressive assertion and a conditional-universal assertion are different assertions even when their matrices are pointwise equivalent. The translations below respect this boundary explicitly.

7.3 Ordinary soundness

Use the pinned repository's scalar theory T_{eq}: ordinary intuitionistic first-order logic with actual equality, the two iterator computations, numerical coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

separation $(0 = 1) \to \bot$, and contextual induction for literal equations. The repository proves its whole-domain bi-interpretability with iPRA.[2]

Define the new ordinary reading recursively by

$$\begin{aligned} [\![s = t]\!] &= (s = t), \\ [\![Q_y E(y)]\!] &= \forall y\,(E(y) \to E(Sy)) \quad (E \text{ a literal equation}), \\ [\![Q_y B(y)]\!] &= [\![B(0)]\!] \to \forall y\,[\![B(y)]\!] \quad (B \text{ compound}), \\ [\![D_{x,y} B]\!] &= \exists x\,[\![Q_y B]\!]. \end{aligned} \tag{59}$$

This translation preserves all free numerical variables and commutes with numerical substitution. It is a finite syntactic translation, not a truth predicate on numerical formula codes.

Theorem 7.5 (Soundness)**.** *If* $\Gamma \vdash_{\mathcal{R}} F$*, then* $[\![\Gamma]\!] \vdash_{T_{\mathrm{eq}}} [\![F]\!]$.

Proof. Hyp, computations, equational transport, and DE have their ordinary meanings. Atomic DI is universal introduction of progress, followed by existential introduction at its witness. Compound DI is conditional universal introduction using numerical coverage, followed by existential introduction.

For atomic QE, retain $H = \forall y(E(y) \to E(Sy))$ and $E(a)$. Equation induction on n gives $E(\mathsf{A}(n, a))$: the base is $E(a)$, and the step uses H at $\mathsf{A}(n, a)$ and IS. The pure equational proofs in Lemma 7.3 also hold in T_{eq}, so commutativity gives $E(\mathsf{A}(a, n))$. For compound QE the minor is $[\![B(0)]\!]$; its conditional-universal major directly supplies the target instance. Thus every rule translates soundly. □

7.4 A precise comparison with the existing native calculus

Let $\mathcal{N}$ denote exactly the current native Eq/Q/D calculus `NPrf`. Its $Q_y B$ always means $B(0) \to \forall y B(y)$; its direct progress rule introduces $Q_y E$ only for equations. Its terms and numerical equality are identical to those of $\mathcal{R}$.

For $E \equiv (s = t)$ define

$$\begin{aligned} C(v, a, b) &:= I(v, a; u.b), \\ G_v E &:= \big(C(v, 0, s) = C(v, 0, t)\big), \\ L(E) &:= D_{r,v} G_v E, \\ P(F) &:= \begin{cases} L(F), & F \text{ an equation}, \\ F, & F \text{ compound}. \end{cases} \end{aligned} \tag{60}$$

All newly displayed binders are fresh; the update in C ignores u, and r is unused in $L(E)$.

Lemma 7.6 (Padding)**.** *In* $\mathcal{R}$*,* $L(E)$ *and* E *are interderivable under arbitrary contexts. Consequently* $P(F)$ *is compound and interderivable with* F*.*

Proof. Computations and equality prove $G_0 E$ and $G_{Sz} E \dashv\vdash E$. From E, introduce $L(E)$ by atomic DI, ignoring its local induction assumption and using E for the successor target. Conversely, DE on $L(E)$ gives $Q_v G_v E$. Supply the theorem $G_0 E$ and apply QE with source 0, initializer 1. I0 gives $G_1 E$, which reduces to E. This proof uses no ordinary logical macro. □

Define a translation $T : \mathcal{N} \to \mathcal{R}$ by

$$\begin{aligned} T(s = t) &= (s = t), \\ T(Q_y B) &= Q_y P(TB), \\ T(D_{x,y} B) &= D_{x,y} P(TB). \end{aligned} \tag{61}$$

It fixes terms, commutes with numerical substitution, and places a compound matrix immediately below every translated binder.

Theorem 7.7 (Translation into the new calculus)**.** *Every $\mathcal{N}$ derivation translates effectively:*

$$\Gamma \vdash_{\mathcal{N}} A \quad \Longrightarrow \quad T\Gamma \vdash_{\mathcal{R}} TA.$$

Proof. Induct on the old derivation. Hyp, computations, and equational transport translate literally. Old QE becomes compound new QE: pad the minor, specialize at the old target using source 0, remove the I0 redex, and unpad. In DE the opened new assumption is exactly $Q_y P(TB)(b, y)$, the translation of the old opened Q assumption. For case DI, unpad the local base, apply the translated premise, and pad the successor conclusion; the new matrix is compound.

For old direct Q-progress at E, the translated target is $Q_y L(E(y))$. Introduce it by compound QI. Its local base $L(E(0))$ supplies $E(0)$. Apply the recovered contextual equation induction to the translated step, then pad $E(Sz)$ to $L(E(Sz))$. This discharges the only nonliteral rule translation. Each construction is finite and preserves eigenvariable conditions after fresh renaming. □

7.5 Logical macros and the order of the completeness proof

The old ordinary logical macros will be denoted $\bot_{\mathcal{N}}$, $\to_{\mathcal{N}}$, $\wedge_{\mathcal{N}}$, $\vee_{\mathcal{N}}$, $\exists_{\mathcal{N}}$, and $\forall_{\mathcal{N}}$. For specificity they are the macros of the pinned `NativeLogic.lean`:[1]

$$\begin{aligned} \bot_{\mathcal{N}} &= (0 = 1), \\ A \to_{\mathcal{N}} B &= Q_y \,\mathrm{sel}(y, A, B), \\ A \wedge_{\mathcal{N}} B &= Q_y \,\mathrm{gate}(y, \mathrm{sel}(\mathrm{par}(y), A, B)), \\ \exists_{\mathcal{N}} x\, A(x) &= D_{x,y} \,\mathrm{gate}(y, A(x)), \\ A \vee_{\mathcal{N}} B &= \exists_{\mathcal{N}} v \ \mathrm{sel}(v, A, B), \\ \forall_{\mathcal{N}} x\, A(x) &= A(0) \wedge_{\mathcal{N}} Q_x A(x). \end{aligned} \tag{62}$$

Here $\mathrm{par}(y) = I(y, 0; u.C(u, 1, 0))$. The old gate acts on equations as G_v in (60). The exact finite recursive definitions of the gate and selector, including their behavior under Q and D, are stated in Appendix 8. The selector has zero endpoint A and successor endpoint B. All these operations concern the *old* uniform conditional calculus. The pinned repository supplies their capture-free definitions and derived rules.

Lemma 7.8 (Logic on image formulas, with arbitrary side context)**.** *For old formulas A, B, the formulas $T(A \to_{\mathcal{N}} B)$, $T(A \wedge_{\mathcal{N}} B)$, and the analogous translated macros have their ordinary natural-deduction rules on principal formulas TA, TB. These derived rules permit an arbitrary retained $\mathcal{R}$ context and arbitrary supplied $\mathcal{R}$ premise proofs. Image existential elimination also permits an arbitrary new conclusion, not necessarily an image formula.*

Proof. Translate the finite *derived-rule constructions*, rather than merely closed old theorems. Their nonlogical steps are replaced by the local simulations in Theorem 7.7. Their open premise

leaves are filled with the supplied new proofs; assumption cut and weakening (Lemma 7.1) retain any additional new context. The construction never examines the syntax of passive side assumptions. Thus this is a uniform rule simulation, not an inference from closed-theorem preservation alone.

For the important existential exception, the old elimination construction opens its defining D by DE, obtains the witness instance from the opened gated closure, and cuts this instance into the supplied minor derivation. It neither analyzes nor introduces the minor's conclusion. The translated construction consequently works for every new conclusion F, using the same freshness condition. This is precisely the structure of the pinned `NPrf.exE` proof. □

We do *not* yet assert that the old selectors or logical macros can be applied unchanged to arbitrary mixed formulas. Such an assertion would incorrectly ignore the root distinction in Q. The next normalization argument first proves that every mixed formula is equivalent to an image.

7.6 Both formula composites for the entire language

Define $J : \mathcal{R} \to \mathcal{N}$, fixing all terms and equations, by

$$\begin{aligned} J(Q_y E) &= \forall_{\mathcal{N}} y\,(E(y) \to_{\mathcal{N}} E(Sy)), && E \text{ an equation},\\ J(Q_y B) &= Q_y J B, && B \text{ compound},\\ J(D_{x,y} E) &= \exists_{\mathcal{N}} x\, \forall_{\mathcal{N}} y\,(E(x,y) \to_{\mathcal{N}} E(x,Sy)), && E \text{ an equation},\\ J(D_{x,y} B) &= D_{x,y} J B, && B \text{ compound}. \end{aligned} \tag{63}$$

This is a fixed finite syntax transformation. It maps compound formulas to compound formulas, preserves free variables, and commutes with numerical substitution. Its ordinary reading in T_{eq} is provably equivalent to (59), by the established meanings of the old logical macros.

Lemma 7.9 (Atomic closure normalization)**.** *For every equation $E(x,y)$, under arbitrary retained contexts,*

$$\begin{aligned} Q_y E(x,y) &\dashv\vdash_{\mathcal{R}} T(\forall_{\mathcal{N}} y(E(x,y) \to_{\mathcal{N}} E(x,Sy))),\\ D_{x,y} E(x,y) &\dashv\vdash_{\mathcal{R}} T(\exists_{\mathcal{N}} x\, \forall_{\mathcal{N}} y(E(x,y) \to_{\mathcal{N}} E(x,Sy))). \end{aligned} \tag{64}$$

Proof. Write $H(x)$ for the image formula on the first line's right. From $Q_y E(x,y)$ and the temporary assumption $E(x,z)$, derived one-step elimination gives $E(x,Sz)$. The image implication and universal introductions of Lemma 7.8 give $H(x)$. Their principal component formulas are just equations, which are already fixed by T; no full mixed-language logical rule has been assumed. Conversely, under $H(x)$ and $E(x,z)$, image universal specialization and implication elimination give $E(x,Sz)$; atomic QI gives $Q_y E(x,y)$.

For D, eliminate the left-hand binder at fresh b, normalize its local $Q_y E(b,y)$ to $H(b)$ by the first part, introduce the image existential at b, and close DE. In the reverse direction, image existential elimination supplies a fresh b and $H(b)$. Under $E(b,z)$ its image universal and implication rules give $E(b,Sz)$, so atomic DI at witness b gives $D_{x,y}E$. Close image existential elimination. Its conclusion is allowed to be this arbitrary new formula by the last assertion of Lemma 7.8. □

Theorem 7.10 (Exact native-language round trips)**.** *For every old formula A and every new formula F,*

$$A \dashv\vdash_{\mathcal{N}} JT(A), \qquad F \dashv\vdash_{\mathcal{R}} TJ(F). \tag{65}$$

These equivalences are uniform under arbitrary additional retained contexts.

Proof. For the old-side composite, note first that $JL(E)$ is old-equivalent to E. It asserts existence of an unused witness and progressivity of the guarded equation G_vE. From E, every positive guard instance follows, so the progress implications hold. Conversely G_0E is a theorem and progress at 0 yields G_1E, hence E. This uses ordinary logical macros in $\mathcal{N}$, whose correctness is already established. Now induct on the old formula. Every binder created by T has a compound matrix, so J gives its old conditional interpretation. The preceding padding equivalence and the induction hypothesis apply under the old Q and D congruence rules.

For the new-side composite, induct on F. Equations are unchanged. Atomic Q and D cases are exactly Lemma 7.9. Suppose the matrix B is compound. The induction hypothesis gives $B \dashv\vdash TJB$. Both B and TJB are compound, since J and T preserve compound roots. Homogeneous congruence (Lemma 7.4) therefore gives

$$QB \dashv\vdash Q(TJB), \qquad DB \dashv\vdash D(TJB).$$

Because TJB is compound, no padding is inserted on these right sides; they are exactly the required TJ composites. There is no appeal to congruence across unlike root classes. The induction proves normalization for every formula before ordinary logic is extended to arbitrary new formulas, avoiding a circular completeness argument. □

All ordinary logical operations in the new language. Now, and only now, define for arbitrary new formulas

$$\begin{aligned} \bot_{\mathcal{R}} &= (0 = 1), \\ F \diamond_{\mathcal{R}} G &:= T(JF \diamond_{\mathcal{N}} JG) \quad (\diamond \in \{\to, \wedge, \vee\}), \\ \exists_{\mathcal{R}} x\, F &:= T(\exists_{\mathcal{N}} x\, JF), \\ \forall_{\mathcal{R}} x\, F &:= T(\forall_{\mathcal{N}} x\, JF). \end{aligned} \tag{66}$$

The round trips convert arbitrary new principal formulas to their images; Lemma 7.8 then supplies the ordinary rules, and the round trips convert their conclusions back. For disjunction or existential elimination with a new conclusion, normalize that conclusion first if necessary. All eigenvariable conditions are preserved because J, T preserve free numerical variables.

This gives intuitionistic natural deduction for the entire new language, including explosion, ordinary numerical coverage, and the quantified rules. It adds no primitive logical axioms. In particular, the system does not use excluded middle for arbitrary quantified formulas.

7.7 Bi-interpretability, consistency, and the inherited arithmetic

Theorem 7.11 (Full consequence equivalence and bi-interpretability)**.** *The translations T, J preserve derivations in both directions and have the round trips* (65)*. The ordinary realization of $\mathcal{R}$ is equiconsistent and bi-interpretable with full first-order iPRA. Both numerical domains are whole, numerical equality is actual equality, and the numerical composite isomorphisms have graph $x = y$.*

Proof. Preservation by T is Theorem 7.7. For J, translate a new derivation into T_{eq} using Theorem 7.5; its formulas are provably equivalent to the ordinary readings of their J translations. The pinned native completeness and native formula round-trip theorems translate this ordinary derivation back into $\mathcal{N}$ and identify each resulting formula with its J translation. Equivalently, one may translate each new rule directly using the old ordinary macros and equation induction; the only added arithmetic calculation is the QE soundness proof above. The two exact formula composites are Theorem 7.10.

The remaining scalar-to-iPRA arithmetic is inherited, not reproved by the new elimination rule. The repository's chain constructs a scalar predecessor, a term reflecting actual equality, quantifier-free induction, pairing, and a compiler for each fixed primitive-recursive description. The reverse interpretation expresses a fixed scalar iterator by primitive recursion. Equation induction proves the two numerical compilation round trips. These are the contents of the pinned `biInterp`, `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, and `pra_native_roundTrip` results.[1, 2]

Composing that existing bi-interpretation with T, J proves the assertion. Our two translations leave every scalar term and the numerical domain unchanged, so they add neither a cut nor a quotient to the existing identity graphs. The designated contradiction $0 = 1$ is fixed. Effective finite proof translations therefore give equiconsistency. This is an external equivalence theorem, not an internal assertion of iPRA's own consistency. □

What changes compared with the earlier progressive proposal. The repository already contains a presentation with a progressive binder on every matrix, separate one-step and equation-iteration eliminations, counter-aware primitive recursion, and an explicit Boolean-cases rule.[9] The progressive reading itself is therefore inherited motivation. The present contribution is the root-dependent complete reading with the scalar iterator unchanged, the single reversed-orbit specialization (51), the removal of a separate case principle, and both round trips for the entire Eq/Q/D formula language. The mixed reading without the stronger elimination would lose information at atomic closures; its soundness alone would not establish the theorem above.

Verification status. The new metatheorems in this section are written proofs. They are not claimed to have been checked in Lean or Rocq. The existing Lean theorem names identify the precise inherited bridge. Executable certificates and checker tests support concrete implementations; they do not replace the universal proof or establish a global lower bound on presentation size.

8 Explicit logical macros in the reference language

This appendix fixes the finite macros used by the interpretations. All operators in this appendix belong to the old conditional language $\mathcal{N}$. They must be expanded there before applying the translation T. Applying these definitions unchanged to arbitrary new mixed closures would be incorrect. These definitions reproduce the interface of `NativeGate.lean` and `NativeLogic.lean` [1].

Write $C(v, a, b) = I(v, a; u.b)$, where u is unused, and $1 = S0$. Bindings below are freshened so that no displayed passive parameter is captured. Define an equation–formula selector $U_v(s, t; B)$ by

$$\begin{aligned} U_v(s, t; r = w) &= (C(v, s, r) = C(v, t, w)), \\ U_v(s, t; Q_y B) &= Q_y U_v(C(y, 0, s), C(y, 0, t); B), \\ U_v(s, t; D_{x,y} B) &= D_{x,y} U_v(C(y, 0, s), C(y, 0, t); B). \end{aligned}$$

Its two endpoints are the equation $s = t$ and the formula B. Put $G_v A = U_v(0, 0; A)$. Then $G_0 A$ is provable and $G_{Sz} A \dashv\vdash A$. A selector between arbitrary formulas is

$$\begin{aligned} M_v(s = t, B) &= U_v(s, t; B), \\ M_v(Q_y A, B) &= Q_y M_v(A, G_y B), \\ M_v(D_{x,y} A, B) &= D_{x,y} M_v(A, G_y B). \end{aligned}$$

This recursion terminates on the depth of its first argument; the separate recursion defining U terminates on its formula argument. For every A, B, the old calculus derives $M_0(A, B) \dashv\vdash A$ and $M_{Sz}(A, B) \dashv\vdash B$.

Let $p(y) = I(y, 0; u.C(u, 1, 0))$. The logical operations are

$$\begin{aligned} \bot_* &= (0 = 1), \\ A \Rightarrow_* B &= Q_y M_y(A, B), \\ A \wedge_* B &= Q_y G_y M_{p(y)}(A, B), \\ \exists_* x\, A(x) &= D_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* v\, M_v(A, B), \\ \forall_* x\, A(x) &= A(0) \wedge_* Q_x A(x). \end{aligned}$$

All new bound variables are fresh. These are definitions by finite syntax, not additional primitive logical axioms or rules.

Why they have the logical rules. Implication introduction uses the zero selector as its local hypothesis and the successor selector as its conclusion; elimination specializes at 1. Existential introduction chooses the witness and uses the successor gate. Existential elimination uses D elimination, the zero-gate theorem, and specialization at 1; its final conclusion can be any formula, independently of the existential matrix.

For conjunction, the gate supplies the base theorem and the computations $p(1) = 1$, $p(2) = 0$ recover the two conjuncts. Numerical cases establish the selector at an arbitrary control from proofs of both alternatives. Disjunction uses witnesses 0 and 1; its elimination uses the derived case rule on an implication to the desired conclusion. The universal macro has its zero instance as one conjunct and every instance as the other conjunct's closure. Finally, $0 = 1$ and the selector computations give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary terms; structural induction on formulas gives falsity elimination.

The full proofs and their substitution lemmas are preserved in the pinned source files included with the package. The new theory's normalization proof uses the translated versions of these derived-rule constructions on T-image principal formulas before it defines the ordinary logical operations on arbitrary new formulas.

9 The arithmetic interface and the inherited bridge

The reference theory T_{eq} is intuitionistic first-order equality logic over scalar iterator terms, with the two computation equations, numerical coverage

$$\forall x\, (x = 0 \vee \exists y\, x = Sy),$$

separation $(0 = S0) \to \bot$, and induction for literal equations under side assumptions independent of the induction variable. Its equality is ordinary equality. The reference iPRA has a symbol for each fixed primitive recursive definition, its defining equations, successor separation and injectivity, and quantifier-free induction over ordinary intuitionistic first-order logic.

The pinned development establishes $\mathcal{N} \simeq T_{\mathrm{eq}} \simeq$ iPRA [2]. The arithmetic part is retained unchanged here. Its substantive stages are:

1. Construct a predecessor in the scalar iterator language. Since the update binds only its previous state, this is a real construction; the counter variable is not silently made available to the update.

2. Derive equality reflection into a term and Boolean operations. Replace a quantifier-free formula by its characteristic equation, which transfers literal-equation induction to quantifier-free induction.
3. Construct pairing and projections, and compile a counter-aware primitive recursion by iterating a state that carries both the counter and the accumulated value.
4. Translate each fixed primitive recursive definition and each scalar iterator term in the opposite direction, proving their defining equations and both composites by equation induction.

The archive contains the corresponding complete written sources `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex` in the pinned reference subtree, together with the ordinary bridge and the native logical basis. These files are included to expose the mathematical dependency; the new report does not claim to have discovered these arithmetic constructions.

There is no quantified code for an arbitrary primitive recursive program and no total universal primitive recursive evaluator in the theory. A compiler acts externally on one finite definition at a time. This distinction matters: adjoining a purported total universal evaluator would change the foundational argument.

Both new translations between $\mathcal{R}$ and $\mathcal{N}$ fix every numerical term and literal equation. Composing them with the reference arithmetic translations therefore retains whole numerical domains and actual equality. The graph of each numerical interpretation composite is provably $x = y$. Formula composites are proved for all formulas, so the conclusion is bi-interpretability, not merely a correspondence of closed equations or an equiconsistency argument.

The precise existing Lean endpoint names are

```
native_iff_pra,  pra_iff_native,
native_roundTrip,  pra_native_roundTrip,
native_inconsistent_iff.
```

Their source was retrieved and authenticated at the pinned commit. Lean/Lake was not available in this run, so these existing files were inspected rather than rebuilt, and no new Lean theorem about $\mathcal{R}$ is claimed.

References

[1] ProveIt, *Native scalar syntax, gates, and logic*, `Native.lean`, `NativeGate.lean`, and `NativeLogic.lean`, at the pinned commit.

[2] ProveIt, *NativeBiInterpretation.lean* and *BiInterpretation.lean*, at the pinned commit. The endpoint sources are retained with the archive.

[3] ProveIt research series, *Complementary introductions with one local assumption*, retained in `reference/foundations/inherited_choice_basis.tex`.

[4] ProveIt research series, scalar predecessor construction, retained in `reference/foundations/scalar_predecessor.tex`.

[5] ProveIt research series, equality reflection, retained in `reference/foundations/equality_reflection.tex`.

[6] ProveIt research series, full primitive recursion, retained in `reference/foundations/primitive_recursion.tex`.

[7] ProveIt, *source_dispatch*, `proofs/zero_source_admissibility.tex`, pinned commit.

[8] ProveIt, *NativeOneIntro.lean*, witness-sensitive single-introduction equivalence theorem, pinned repository.

[9] ProveIt, *Research/research/theory/progressive_binder.tex*, the earlier progressive-binder development cited by the inherited report.

[10] ProveIt incoming research, *A uniform introduction rule for intuitionistic primitive recursive arithmetic*, archive `ipra_uniform_progress_2026-11-11.zip`.

[11] V. Reshetnikov and ProveIt contributors, *ConciseIPRA*. https://github.com/VladimirReshetnikov/ProveIt/tree/76697262071ada91f29c14d9cf3801a64dacb994/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.